

THREAT INTELLIGENCE INTEGRATION WITH OPENCTI

Project Details	Information
Name	Ashogbon Ayomikun Abdulazeez
Project	Threat Intelligence Integration
Date	September 2026

1. Executive Summary

This project extended an existing Security Operations Center (SOC) homelab by adding OpenCTI as a threat intelligence platform to provide additional context for security monitoring and investigation. The existing environment included a Wazuh-based SIEM, Windows and Ubuntu endpoints, Sysmon, Suricata, and a pfSense firewall. OpenCTI was deployed using Docker and integrated with three external threat intelligence sources: URLhaus, MalwareBazaar, and ThreatFox, while MITRE ATT&CK was also integrated to provide structured threat intelligence and technique information. A Wazuh–OpenCTI enrichment pipeline was then implemented to allow security events containing Indicators of Compromise (IOCs) to be checked against OpenCTI. The integration was validated using both non-matching and matching IOC tests and through a simulated brute-force activity, demonstrating that threat intelligence could be incorporated into the existing SOC monitoring and alert investigation workflow.

2. Scope & Objective

The scope of this project was to extend the existing SOC homelab by integrating OpenCTI with the Wazuh-based security monitoring environment. The primary objective was to enrich security alerts generated by monitored endpoints with relevant threat intelligence, allowing analysts to obtain additional context about Indicators of Compromise (IOCs) such as IP addresses, URLs, and file hashes.

The intended enrichment workflow was designed to allow an IOC contained in a Wazuh alert to be checked against threat intelligence available in OpenCTI. Where a matching IOC is identified, the associated threat intelligence can provide additional context to support the analyst's investigation and determine whether the activity is associated with known malicious infrastructure or artifacts. This approach reduces the need for analysts to manually investigate every IOC using separate threat intelligence sources.

The project therefore focused on establishing and demonstrating the workflow from endpoint telemetry to Wazuh, from Wazuh to OpenCTI, and from OpenCTI back to the analyst through enriched security alerts. The project also included the integration of external threat intelligence feeds, MITRE ATT&CK data, and the application of threat intelligence to an existing brute-force detection use case.

3. Methodology

3.1 Existing SOC Environment

Before integrating OpenCTI, the project was built on an existing SOC homelab that had already been deployed for security monitoring, log collection, and detection engineering. The central security monitoring platform was Wazuh, deployed through Docker on a Kali Linux virtual machine. The Wazuh environment served as the primary location for collecting, processing, and reviewing security events generated across the lab.

The lab included a Linux Ubuntu endpoint configured with a Wazuh Agent. Multiple sources of security telemetry were configured to send logs to Wazuh, including audit logs, system logs, journald logs, Syslog, and Suricata network security logs. Suricata was deployed on the Ubuntu endpoint to provide network-related security visibility, while its generated logs were forwarded through the Wazuh Agent for centralized monitoring.

A Windows endpoint was also deployed as part of the environment. Sysmon was installed on the Windows endpoint to provide additional system-level telemetry and improve visibility into activities occurring on the system. The resulting Windows security and Sysmon-related events were collected by the Wazuh Agent and forwarded to the Wazuh environment for analysis.

The environment also included a pfSense firewall, which provided network traffic control and firewall visibility within the lab. The firewall was configured to forward its logs to Wazuh using UDP port 514, allowing firewall events to be incorporated into the centralized monitoring environment.

In addition to log collection, the existing Wazuh environment contained seven custom detection rules designed to identify different types of suspicious activity. These included a brute-force detection rule, which triggers when multiple failed login attempts are observed within the defined time period; a suspicious process execution rule for activity involving suspicious locations such as the /tmp directory; and a cron job detection rule designed to identify the creation or modification of cron jobs that could be used for persistence.

The remaining detection rules focused on credential and execution-related activity. These included rules for OS credential dumping, unsecured credential access, and Unix script interpreter activity, with the latter monitoring interactive execution involving interpreters such as Python and Bash. A system service detection rule was also implemented to identify service-related activity that could potentially be used for persistence.

This existing environment provided the security telemetry and detection capabilities required for the OpenCTI integration. Rather than building a separate SOC environment for the project, OpenCTI was introduced as an additional threat intelligence layer on top of the existing infrastructure.

3.2 Phase 1 — Deploy OpenCTI and Get Oriented

The first phase of the project focused on deploying OpenCTI and becoming familiar with its platform, data model, and core functionality before integrating external threat intelligence sources or connecting it to the existing Wazuh environment.

OpenCTI was deployed using Docker Compose directly on the host operating system. The deployment included the core OpenCTI platform and its supporting services, including Elasticsearch, Redis, RabbitMQ, and MinIO, together with the required connector components. Deploying the platform on the host system provided a dedicated environment in which OpenCTI could be configured and explored without introducing another virtual machine into the existing SOC infrastructure.

After the deployment was completed, an analyst account was created and the basic organizational settings were configured. The OpenCTI interface and data model were then explored to develop an understanding of how threat intelligence is represented and related within the platform. This included reviewing Observables, Indicators, Reports, Threat Actors, Campaigns, Malware, and Intrusion Sets.

The MITRE ATT&CK connector was subsequently enabled to populate the OpenCTI knowledge base with MITRE ATT&CK information. The connector initially required troubleshooting before the data became available and browsable within the platform. After the issue was resolved, MITRE ATT&CK information became available for use within the OpenCTI environment.

As an additional validation of the platform's write capabilities, a Threat Actor entity and an Indicator were manually created. A relationship between intelligence objects was also simulated and confirmed successfully.

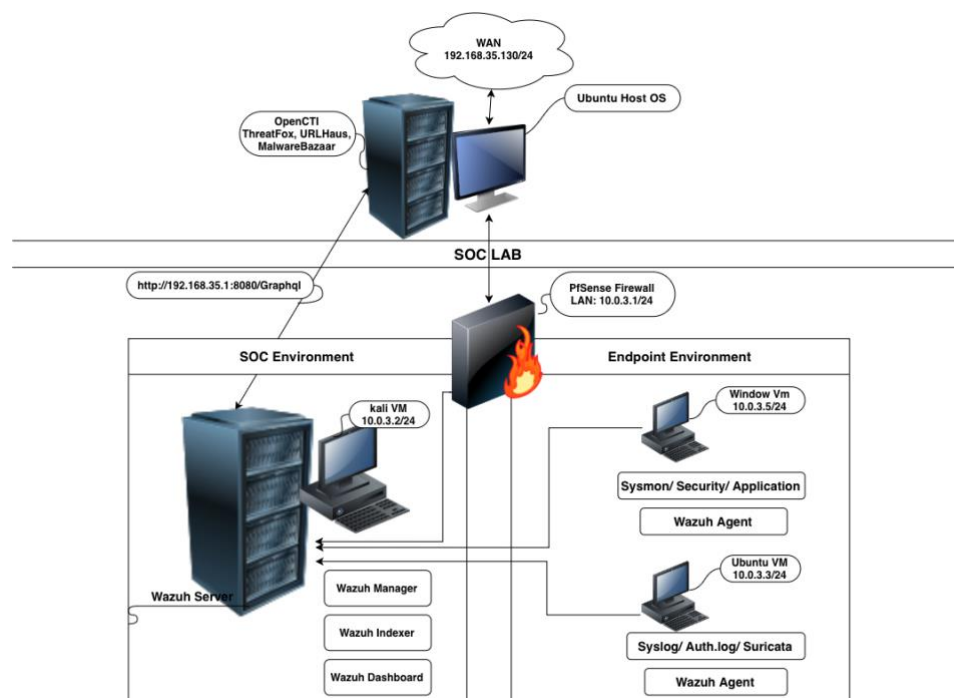
These activities established OpenCTI as the threat intelligence component of the existing SOC environment and provided the foundation for the subsequent integration of external threat intelligence feeds.

3.2.1 Lab Architecture and Data Flow

The completed lab architecture consisted of the existing Wazuh monitoring environment, endpoint telemetry sources, network security monitoring, external threat intelligence feeds, and OpenCTI.

The architecture included an Ubuntu host running the OpenCTI environment, the Wazuh Server and its Manager, Indexer, and Dashboard components, a pfSense firewall, a Windows endpoint with Sysmon and the Wazuh Agent, and an Ubuntu endpoint with Syslog, authentication logs, Suricata, and the Wazuh Agent.

External threat intelligence was provided through URLhaus, MalwareBazaar, and ThreatFox. OpenCTI served as the central platform for collecting and organizing this intelligence. Wazuh remained responsible for collecting and analyzing security events from the monitored environment, while OpenCTI provided additional threat intelligence context when relevant indicators were identified.



The architecture demonstrates the intended flow of information between endpoint telemetry, the SIEM, threat intelligence sources, OpenCTI, and the analyst.

3.3 Phase 2 — Threat Intelligence Feed Integration

The second phase focused on connecting external threat intelligence feeds to OpenCTI and verifying that useful indicators were being imported.

The initial feed configuration included URLhaus, MalwareBazaar, and AlienVault OTX. Docker Compose configurations were used to deploy the connectors alongside the OpenCTI environment.

The URLhaus connector successfully populated OpenCTI with threat intelligence. MalwareBazaar initially failed to deliver data because of a configuration issue involving the OpenCTI service address. The configuration was subsequently corrected, after which MalwareBazaar successfully delivered indicators into OpenCTI.

AlienVault OTX was also tested during the project. The connector initially produced an unsupported time-format error. After the time format was adjusted, the connector subsequently returned a 503 server error. Because the connector continued to fail, AlienVault OTX was removed and ThreatFox was deployed instead. ThreatFox successfully populated OpenCTI.

The final working external intelligence sources were therefore URLhaus, MalwareBazaar, and ThreatFox.

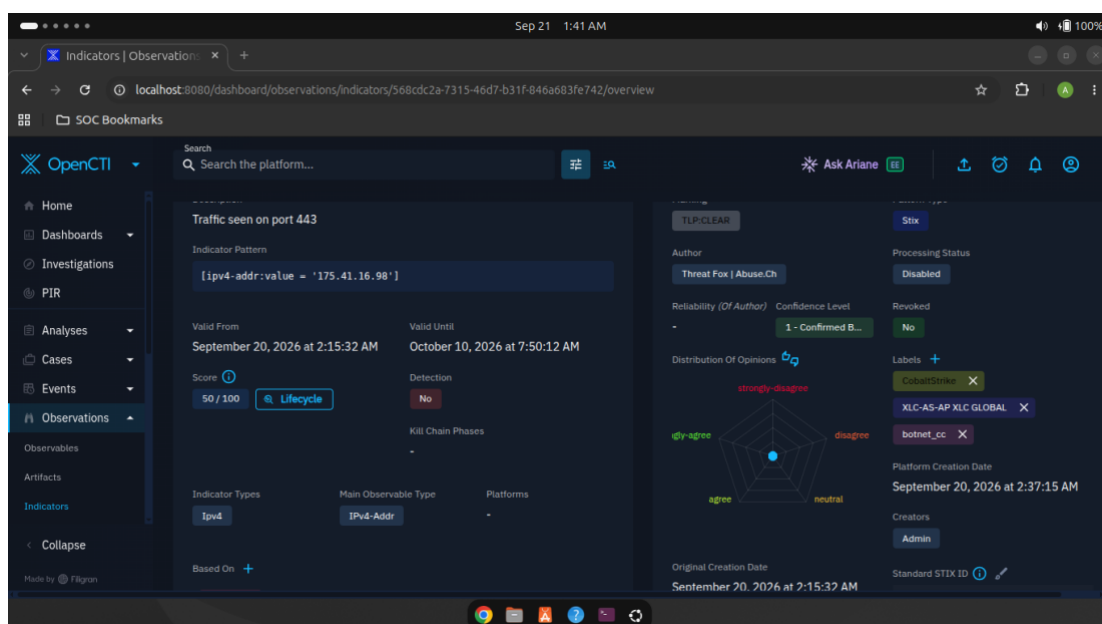
The imported intelligence included different IOC types, including IP addresses, URLs, and file hashes. The OpenCTI interface was used to review information associated with the indicators, including source information, confidence, TLP, and relationships with MITRE ATT&CK information.

The available threat intelligence was also filtered and searched for specific malware-related information, including Mirai-related intelligence.

Three indicators were selected for closer review because they provided useful examples of the types of intelligence that could be used during investigation.

3.3.1 ThreatFox Indicator — 175.41.16.98

The IP address 175.41.16.98 was identified through ThreatFox and was associated with labels including CobaltStrike and botnet_cc. The indicator was identified as active Cobalt Strike command-and-control infrastructure, with the confidence described as confirmed by other sources.



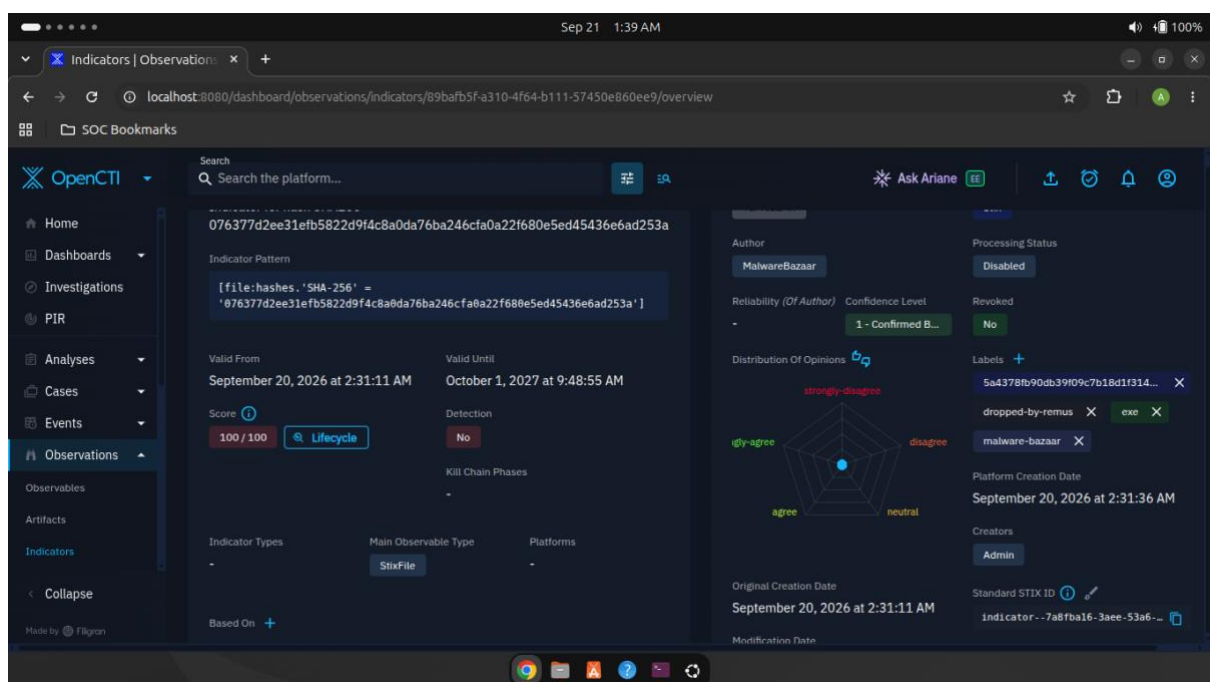
This indicator was relevant to the lab because an outbound connection from a monitored endpoint to this address could warrant further investigation and could be enriched with threat intelligence from OpenCTI.

3.3.2 MalwareBazaar SHA-256 Indicator

The SHA-256 indicator:

076377d2ee31efb5822d9f4c8a0da76ba246cfa0a22f680e5ed45436e6ad253a

was sourced from MalwareBazaar. The indicator had a score of 100/100 and was associated with a Windows executable and the Remus loader malware family.



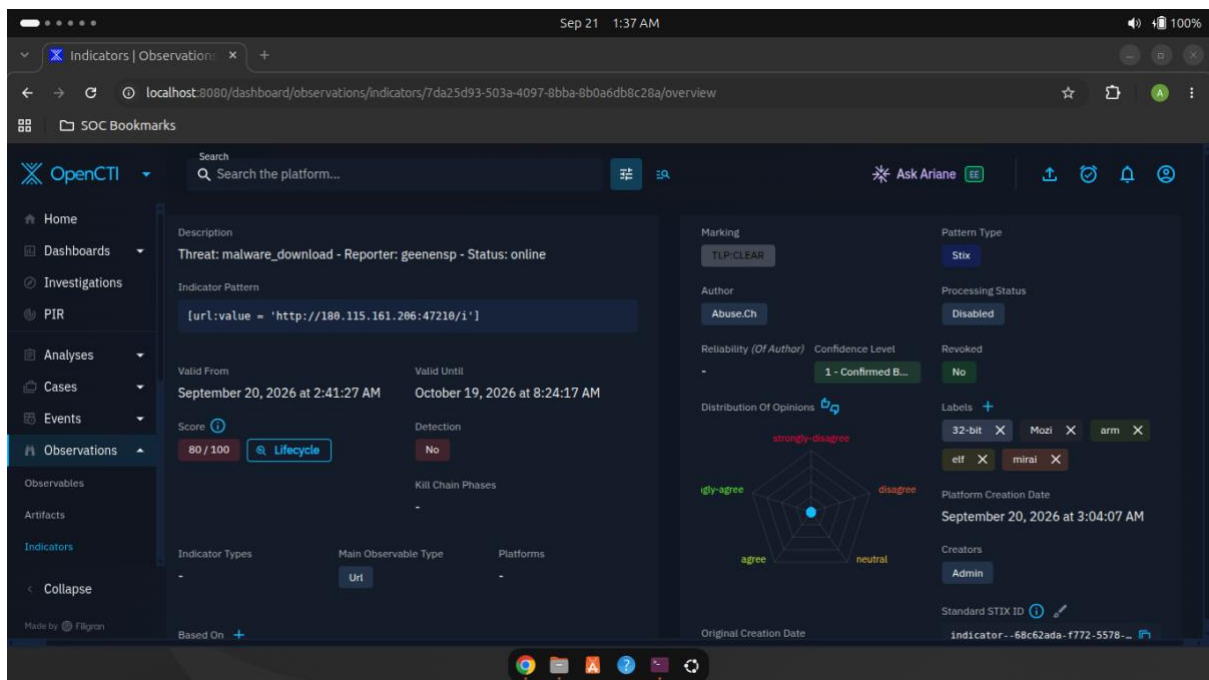
This indicator demonstrated how file-hash intelligence from MalwareBazaar could provide additional context for suspicious files observed on a Windows endpoint.

3.3.3 URLhaus Indicator — 180.115.161.206

The URL:

<http://180.115.161.206:47210/i>

was sourced from Abuse.ch URLhaus and had a score of 80/100. It was identified as an actively online malware distribution point serving ELF ARM binaries and was associated with labels including Mozi and Mirai.



This indicator provided an example of how URLhaus intelligence could be relevant to the Linux side of the SOC environment.

3.3.4 Mirai Intelligence Search

Threat intelligence related to Mirai was also searched and reviewed within OpenCTI to demonstrate the ability to filter and investigate intelligence associated with a particular malware family.

3.4 Phase 3 — SIEM ↔ OpenCTI Enrichment Pipeline

The third phase focused on connecting Wazuh to OpenCTI so that security events containing potential IOCs could be checked against the threat intelligence platform.

A GraphQL-based integration approach was selected. A custom OpenCTI integration was obtained and installed within the Wazuh integrations directory. The integration was configured to communicate with the OpenCTI GraphQL endpoint.

The integration was then connected to Wazuh through the Wazuh Manager configuration. Custom Wazuh rules were created to process the responses from OpenCTI and distinguish between matched malicious indicators and indicators that were known to OpenCTI but were not flagged as malicious.

The integration was tested manually before being validated through the live event-driven Wazuh process.

A negative IOC test was performed using an IOC that did not produce a match in OpenCTI. The integration completed successfully and returned zero matches.

A positive test was then performed using 37.187.150.121. OpenCTI successfully identified a matching IPv4 indicator and returned a score of 80 with seven indicators associated with the match.

The integration was subsequently tested using a real Wazuh security event generated by simulated brute-force activity. Initially, the background integration process was difficult to troubleshoot because the Wazuh integration debugging level was set to 0. After increasing the debugging level and restarting the Wazuh Manager, the background process became visible in the logs and the event-driven pipeline could be confirmed.

The completed workflow was:

Security Event → Wazuh → OpenCTI IOC Lookup → Threat Intelligence Match → Enriched Wazuh Alert → Analyst Investigation

This demonstrated that the integration was not limited to manually executing the enrichment script and could operate as part of the Wazuh event-driven monitoring process.

3.5 Phase 4 — Applying Threat Intelligence to Detection

The fourth phase focused on applying the OpenCTI enrichment capability to an existing detection rule.

3.5.1 Selected Detection Rule

The selected use case was the existing Wazuh brute-force detection rule.

The rule was designed to detect multiple failed login attempts occurring within the defined period. This detection was selected because it provided a practical opportunity to demonstrate how threat intelligence could add context to an otherwise suspicious authentication event.

3.5.2 Detection Behavior After OpenCTI Integration

After the OpenCTI integration was added, a brute-force event could trigger the existing Wazuh detection and then be processed by the OpenCTI integration.

Where the source IP matched threat intelligence in OpenCTI and satisfied the relevant malicious-match conditions, an additional OpenCTI alert could be generated to provide threat intelligence context.

Where the source IP did not produce a malicious OpenCTI match, the corresponding OpenCTI malicious alert was not generated. The original brute-force detection remained separate from the OpenCTI enrichment result.

3.5.3 Brute-Force Simulation

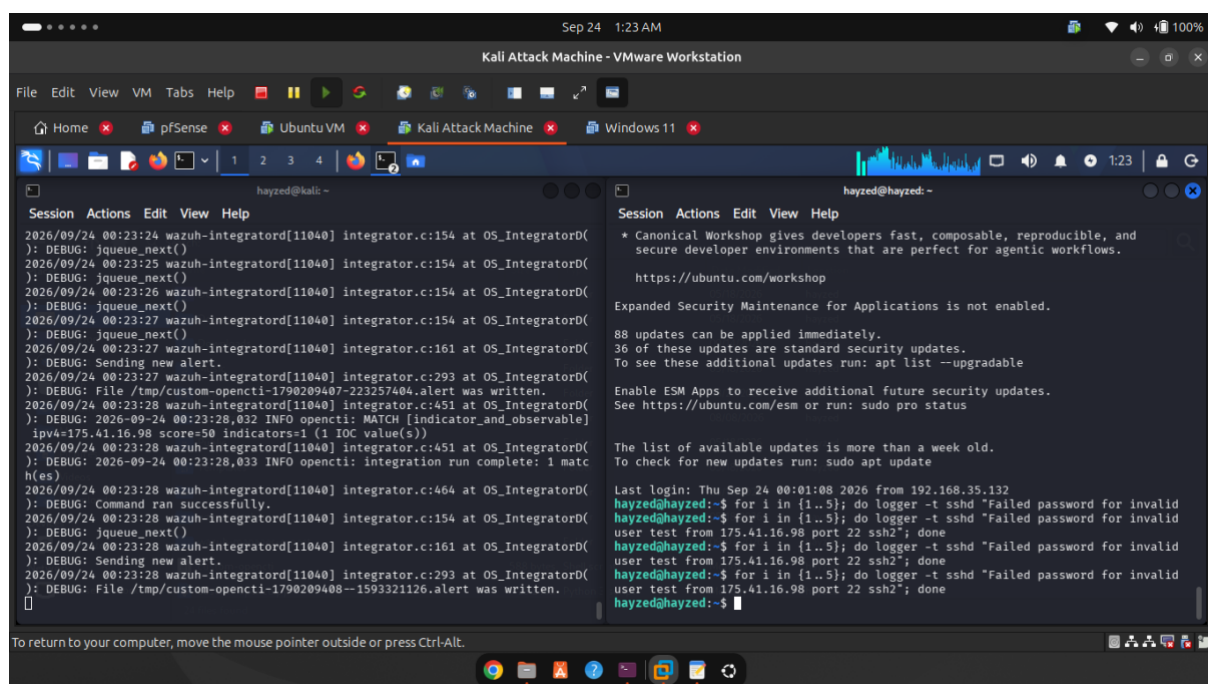
The brute-force scenario was simulated on the monitored endpoint using five failed SSH authentication events originating from the known IOC 175.41.16.98.

The following command was used:

```
for i in {1..5}; do logger -t sshd "Failed password for invalid user test from 175.41.16.98 port 22 ssh2"; done
```

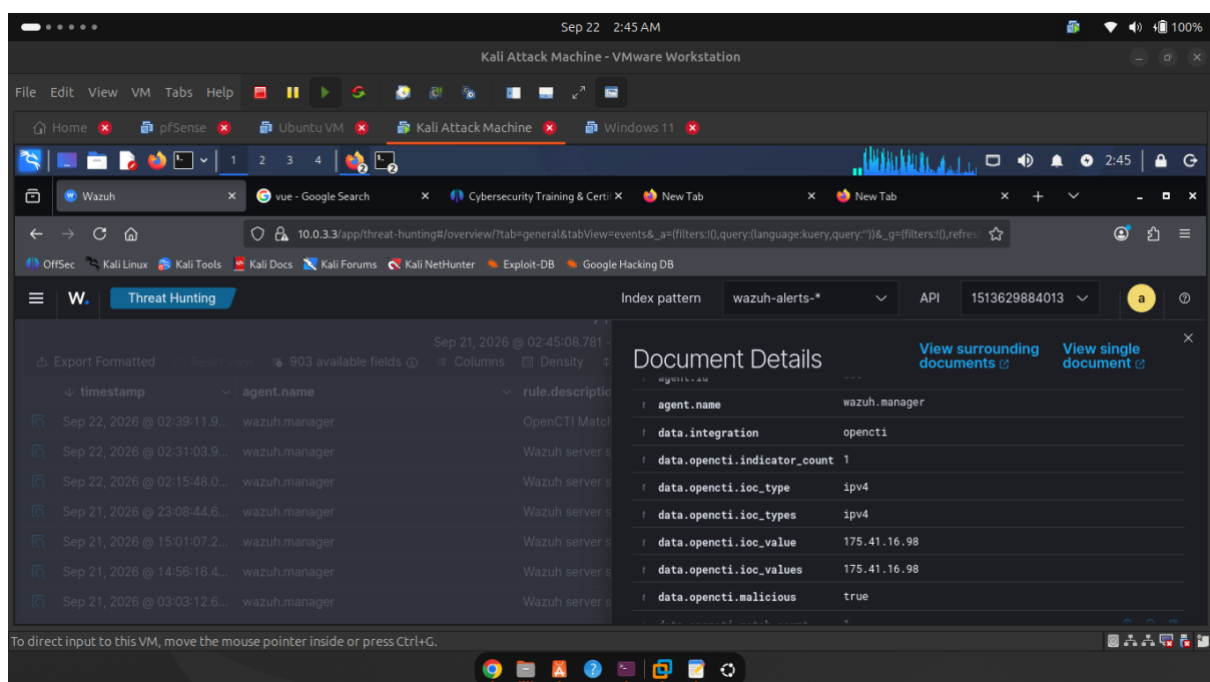
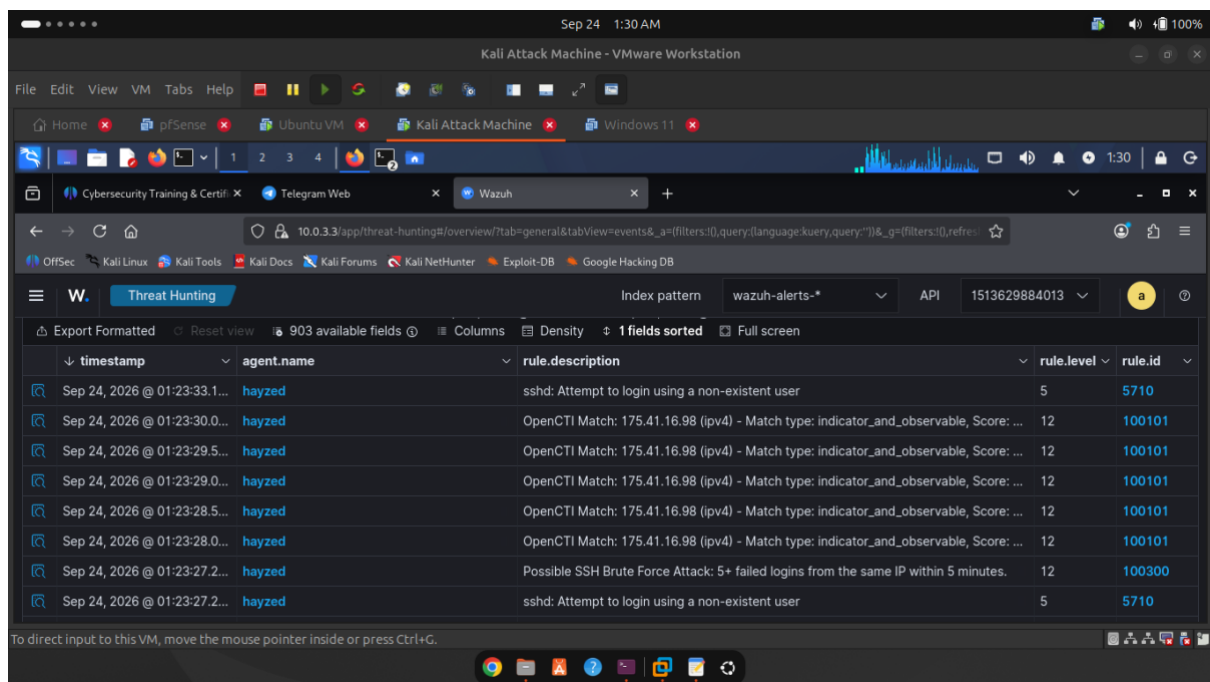
3.5.4 Monitoring and Alert Validation

The generated activity was monitored through the Wazuh integration process. The event was successfully intercepted by the background Wazuh integration process, processed through the OpenCTI enrichment workflow, and displayed within the Wazuh environment.



This evidence contains the bruteforce simulation(3.5.3), monitoring and alert validation (3.5.4)

The resulting alert was visible on the Wazuh dashboard.



3.5.5 Phase 4 Validation Result

The brute-force simulation demonstrated the practical application of threat intelligence to an existing detection. Wazuh identified the authentication activity, while OpenCTI provided additional intelligence about the source IOC.

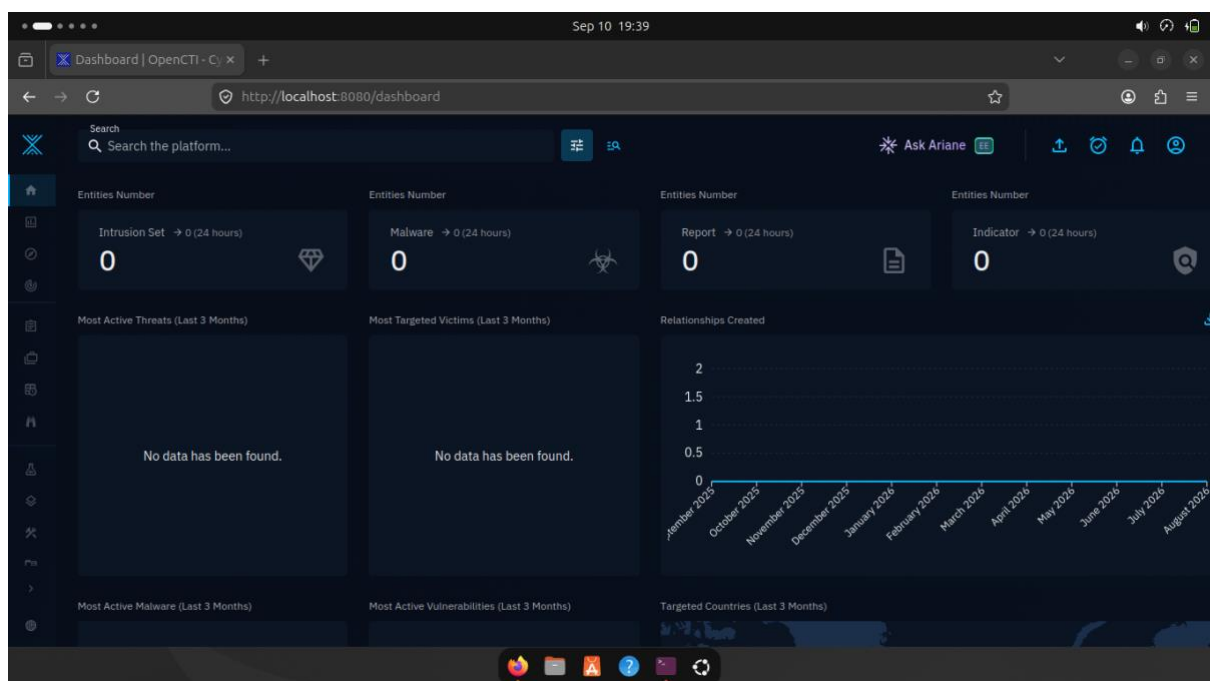
This demonstrated that the threat intelligence layer could be used to provide additional context during alert triage rather than relying only on the original security event.

4. Findings

4.1 OpenCTI Deployment and Platform Validation

The OpenCTI platform was successfully deployed and made accessible for threat intelligence management.

An analyst account was created and used to interact with the platform.

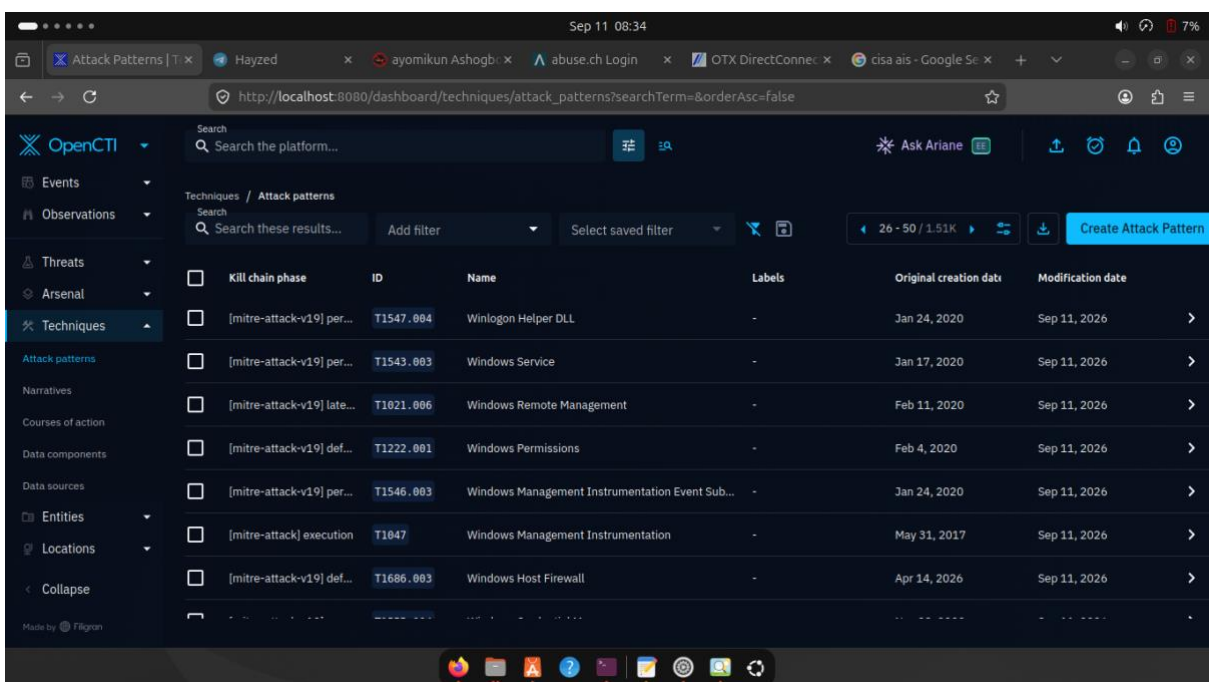
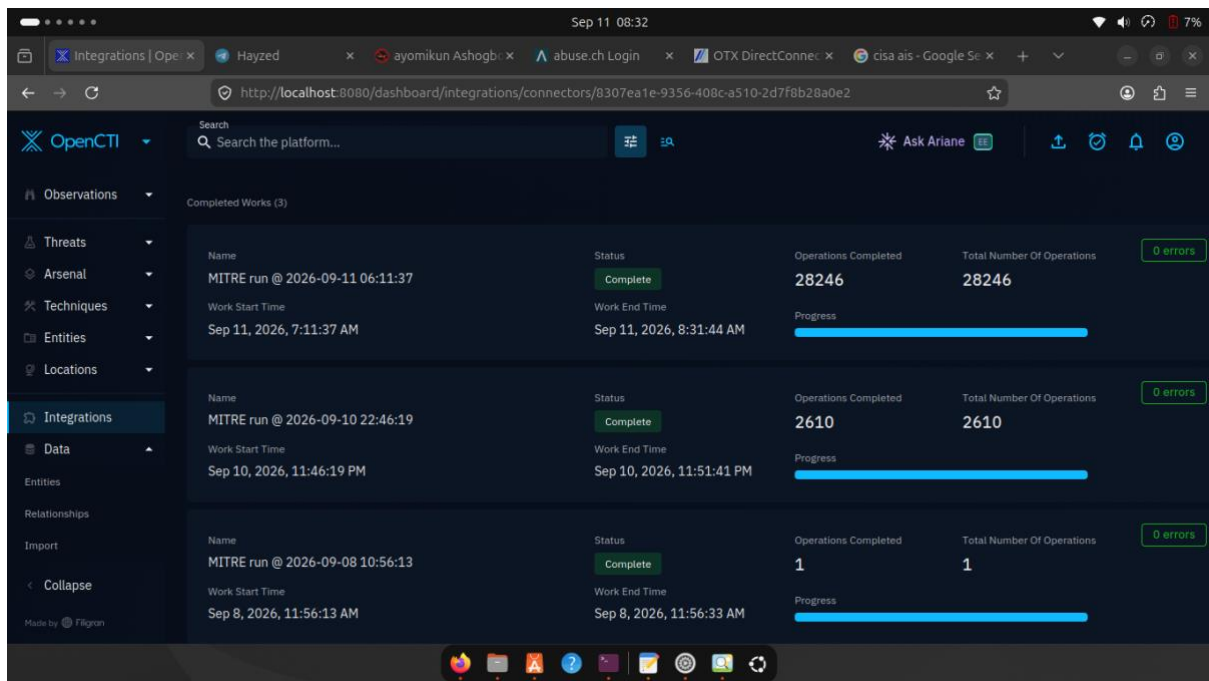


Opencti Deployed

The screenshot shows the 'Users' management page in the OpenCTI dashboard. The page has a sidebar with navigation options like Integrations, Data, Settings, Parameters, Security, Customization, Taxonomies, Activity, File indexing, and Filigran Experience. The main content area displays a table of users with columns for Name, Email, Firstname, Lastname, Max Confidei, 2FA, Password valid until, and Platform creat. There are also buttons for 'Add filter', 'Select saved filter', and 'Create User'.

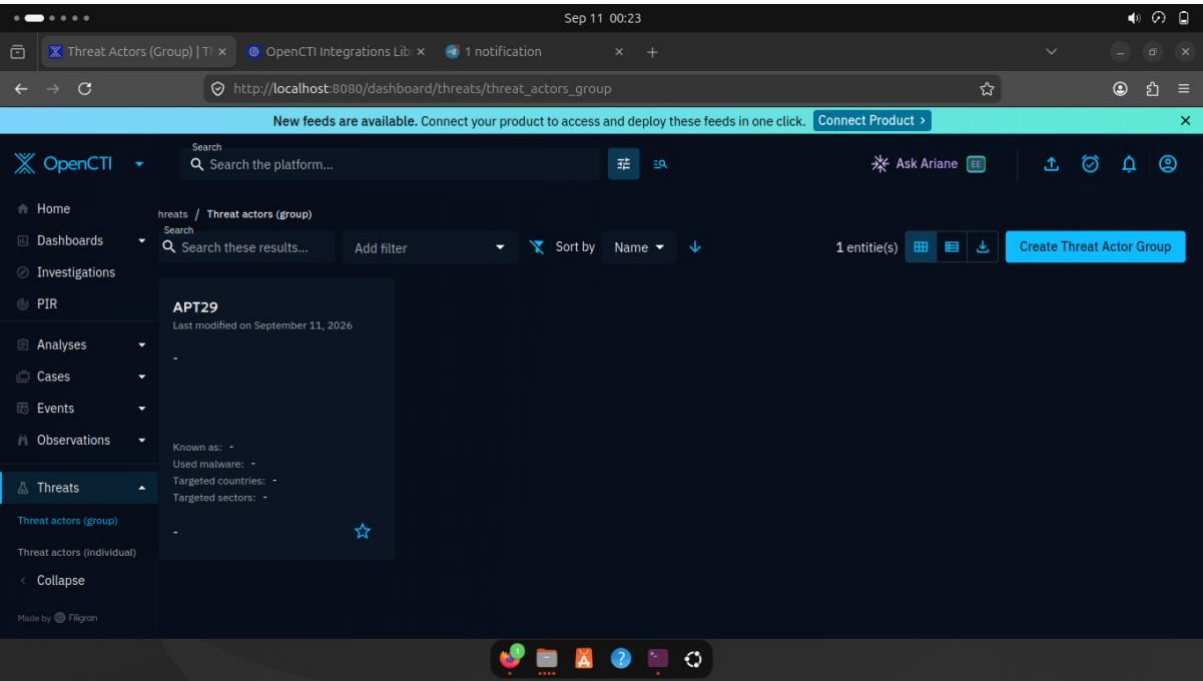
Name	Email	Firstname	Lastname	Max Confidei	2FA	Password valid until	Platform creat
hayzed	knghayzed63@...	Ayomikun	Ashogbon	100	-	Sep 10, 2026	
[C] MITRE ATT&...	mitre-att-ck@co...			75	-	Sep 8, 2026	
[C] OpenCTI Dat...	opencti-dataset...			100	-	Sep 8, 2026	
admin	khayzed101@g...	Admin	OpenCTI	100	-	Sep 8, 2026	

The MITRE ATT&CK connector was enabled and, after troubleshooting, successfully populated the OpenCTI environment with MITRE ATT&CK information.

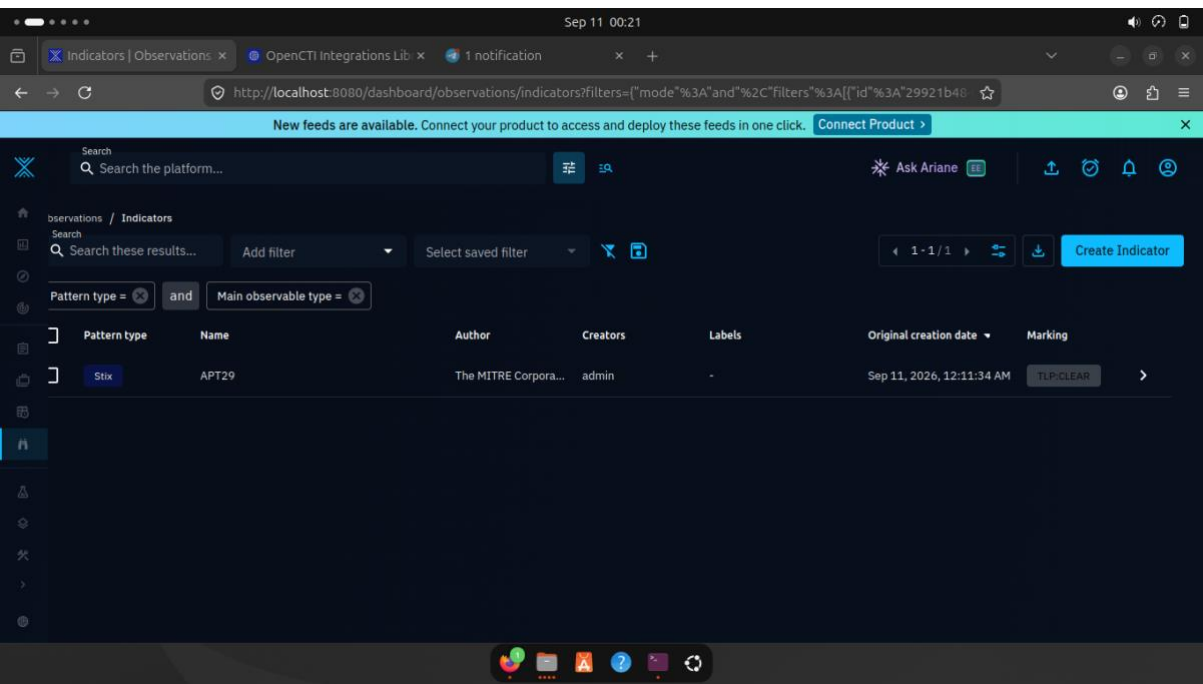


The platform was used to explore different threat intelligence entities, including Threat Actors, Indicators, Observables, Malware, Campaigns, Reports, and Intrusion Sets.

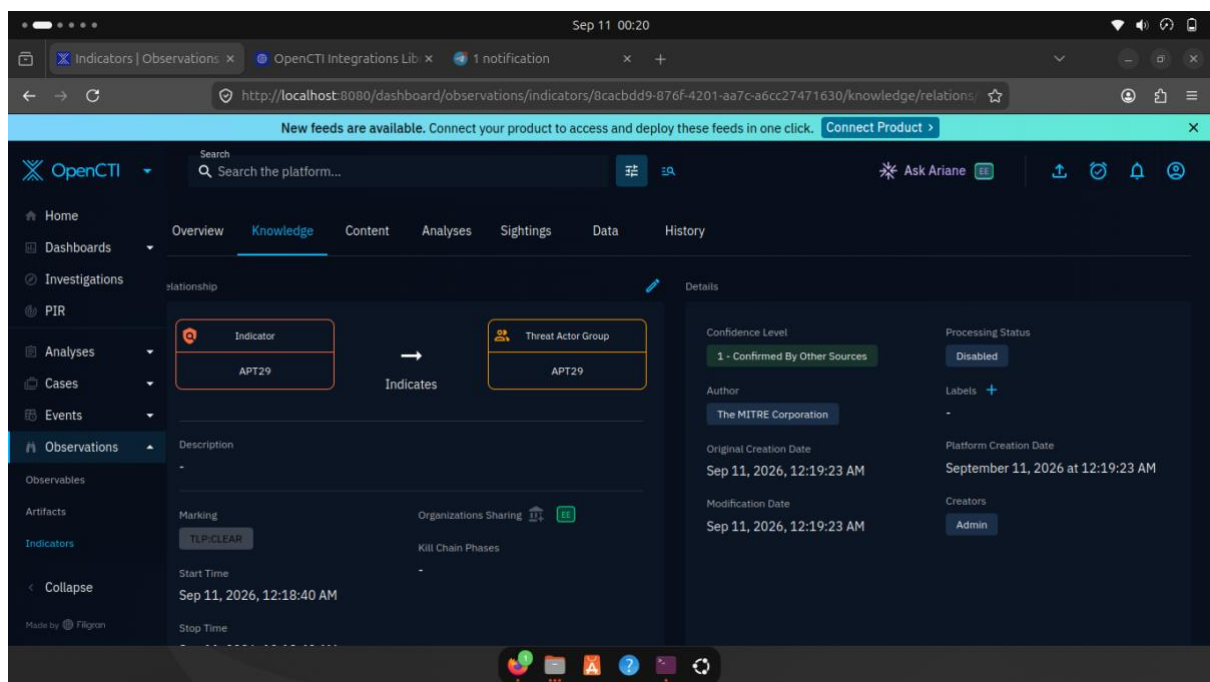
A Threat Actor was manually created as part of the platform validation process.



A manual Indicator was also created.



The relationship between intelligence objects was then simulated and confirmed.



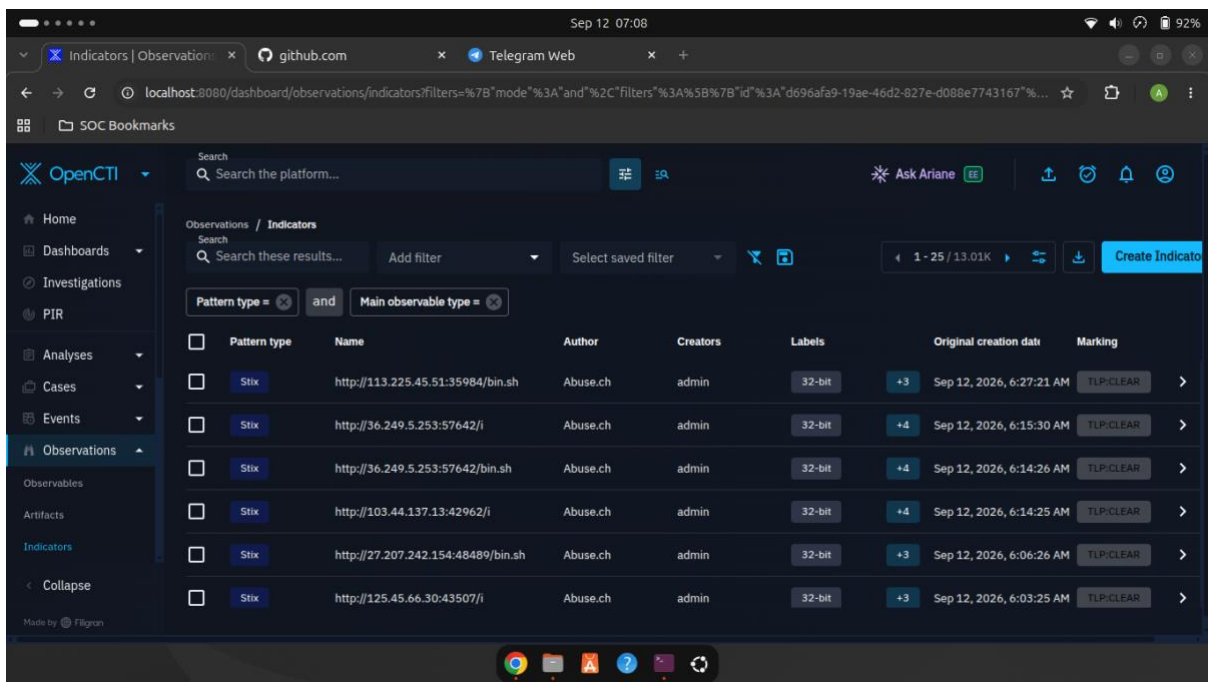
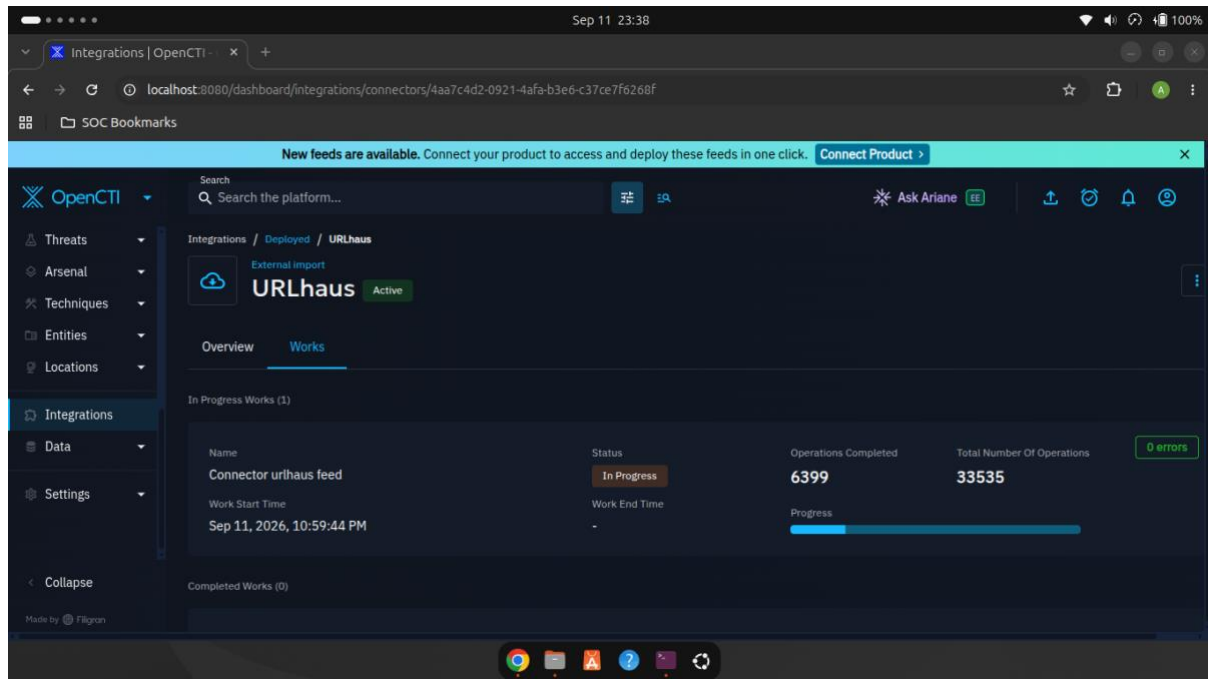
These activities confirmed that the OpenCTI platform was functioning and that intelligence objects could be created and related within the environment.

4.2 External Threat Intelligence Feed Findings

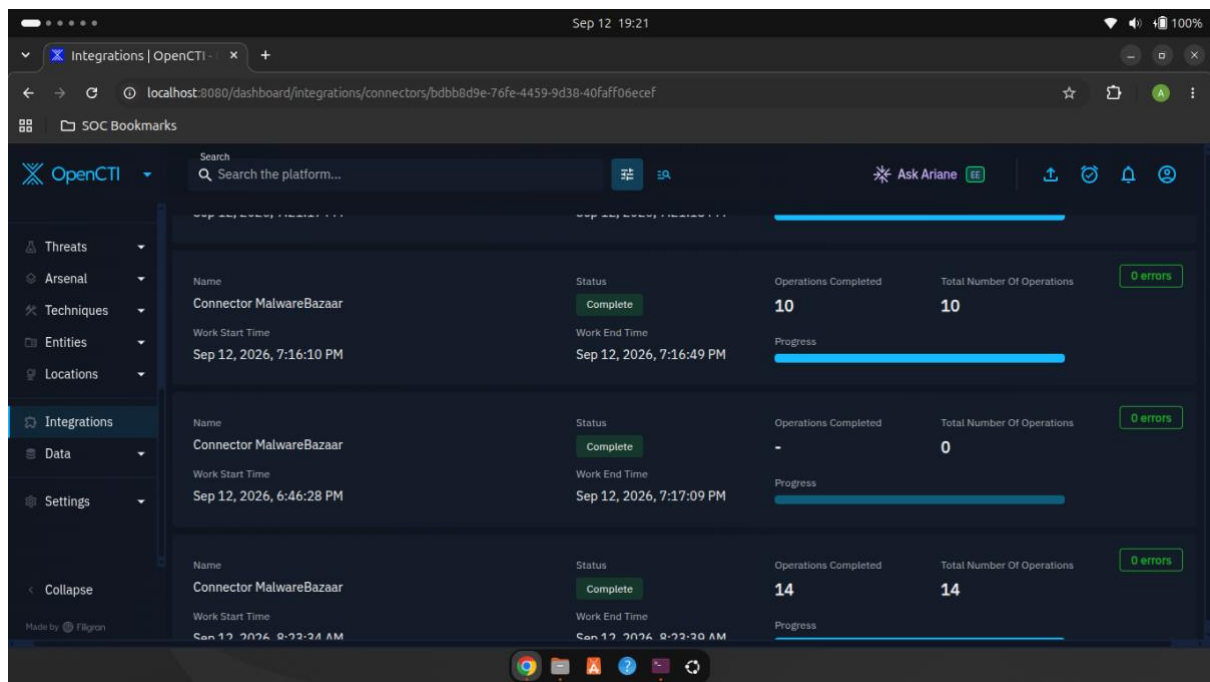
Three external threat intelligence feeds were successfully integrated into the final environment:

- URLhaus
- MalwareBazaar
- ThreatFox

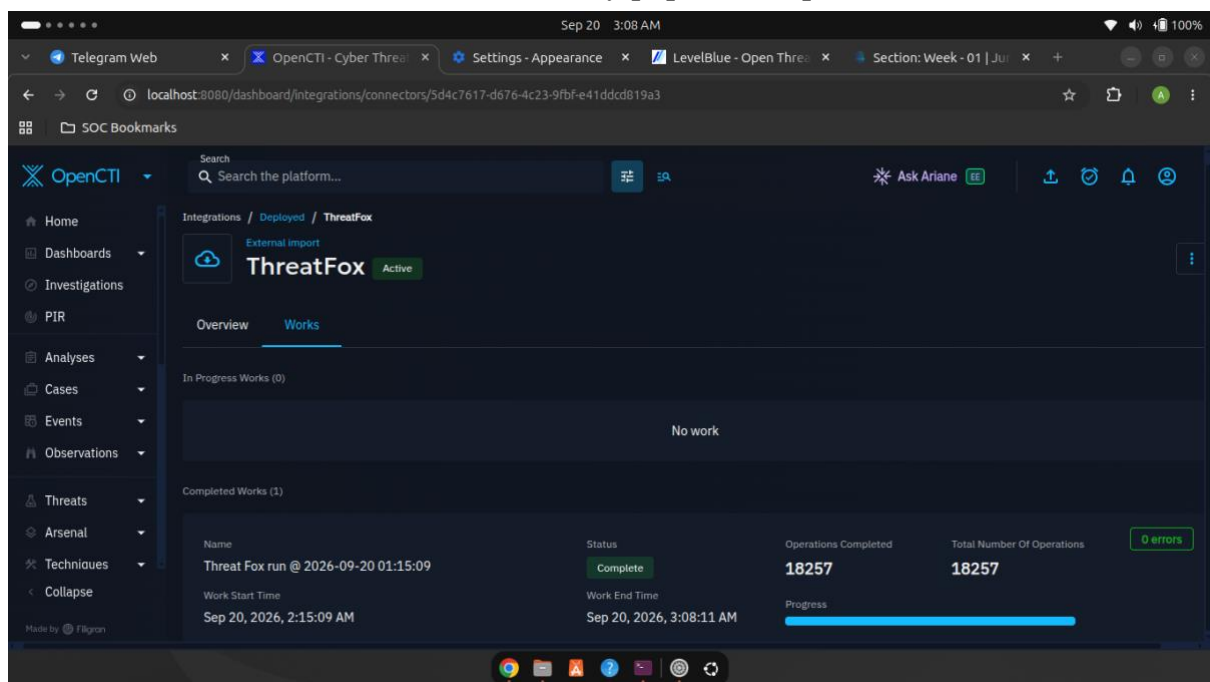
URLhaus successfully populated OpenCTI with threat intelligence.



MalwareBazaar was successfully integrated after its configuration was corrected



ThreatFox was subsequently deployed as the replacement for the unsuccessful AlienVault OTX connector and successfully populated OpenCTI.

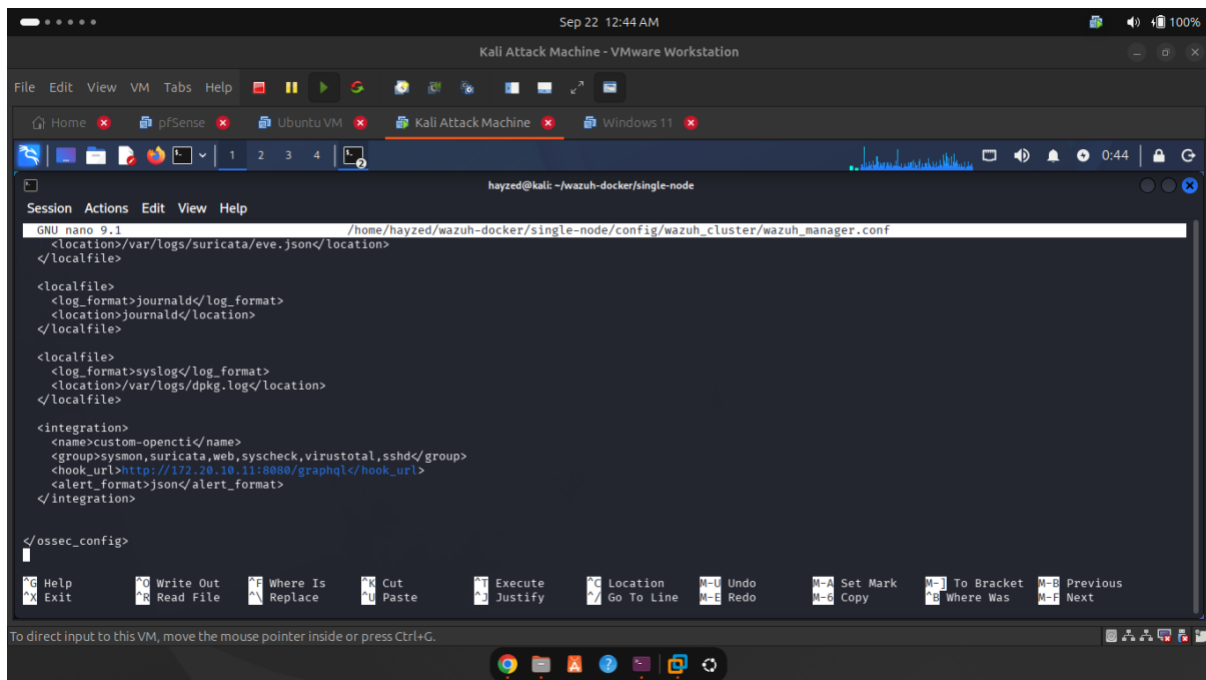


The imported intelligence included IP addresses, URLs, and file hashes. The indicators could be reviewed according to their source, confidence, TLP information, and relationships with other threat intelligence.

The three selected indicators demonstrated how the feeds could provide intelligence relevant to different parts of the SOC environment.

4.3 Wazuh–OpenCTI Integration Findings

The custom Wazuh–OpenCTI integration was successfully installed and configured.



```
GNU nano 9.1 /home/hayzed/wazuh-docker/single-node/config/wazuh_cluster/wazuh_manager.conf
<location>/var/logs/suricata/eve.json</location>
</localfile>

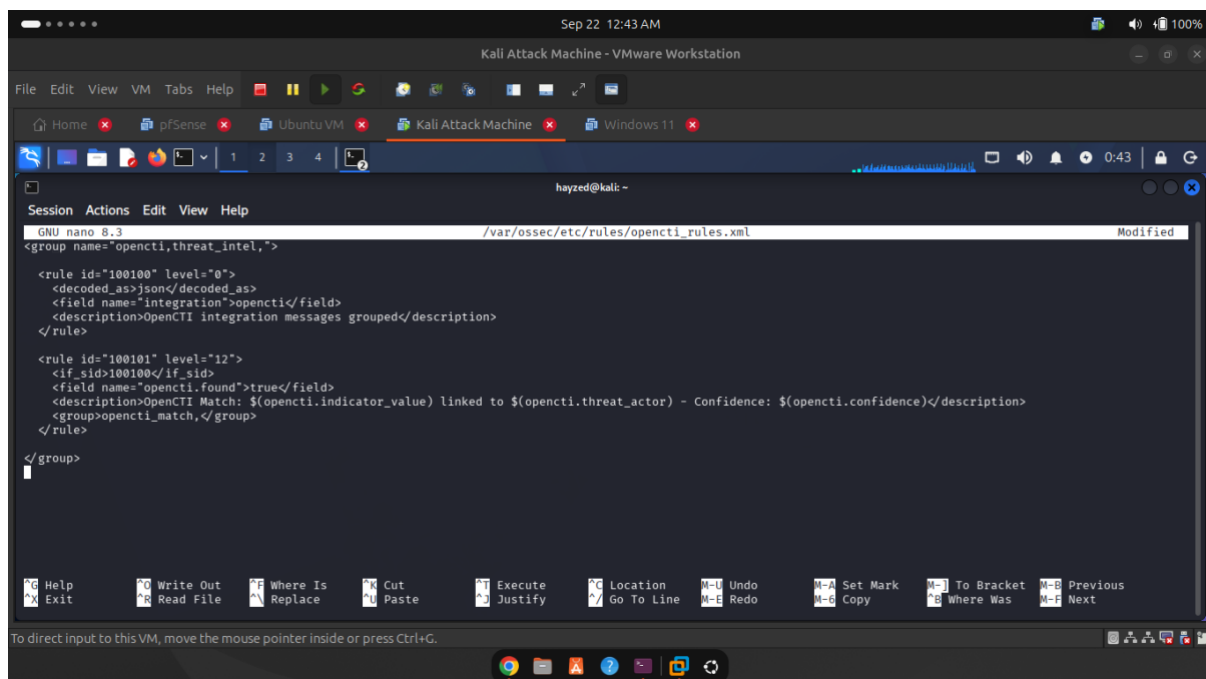
<localfile>
  <log_format>journald</log_format>
  <location>journald</location>
</localfile>

<localfile>
  <log_format>syslog</log_format>
  <location>/var/logs/dpkg.log</location>
</localfile>

<integration>
  <name>custom-opencti</name>
  <group>sysmon,suricata,web,syscheck,virustotal,sshd</group>
  <hook_url>http://172.20.10.11:8080/graphql</hook_url>
  <alert_format>json/alert_format</alert_format>
</integration>

</ossec_config>
```

Custom Wazuh rules were created to process OpenCTI integration messages.

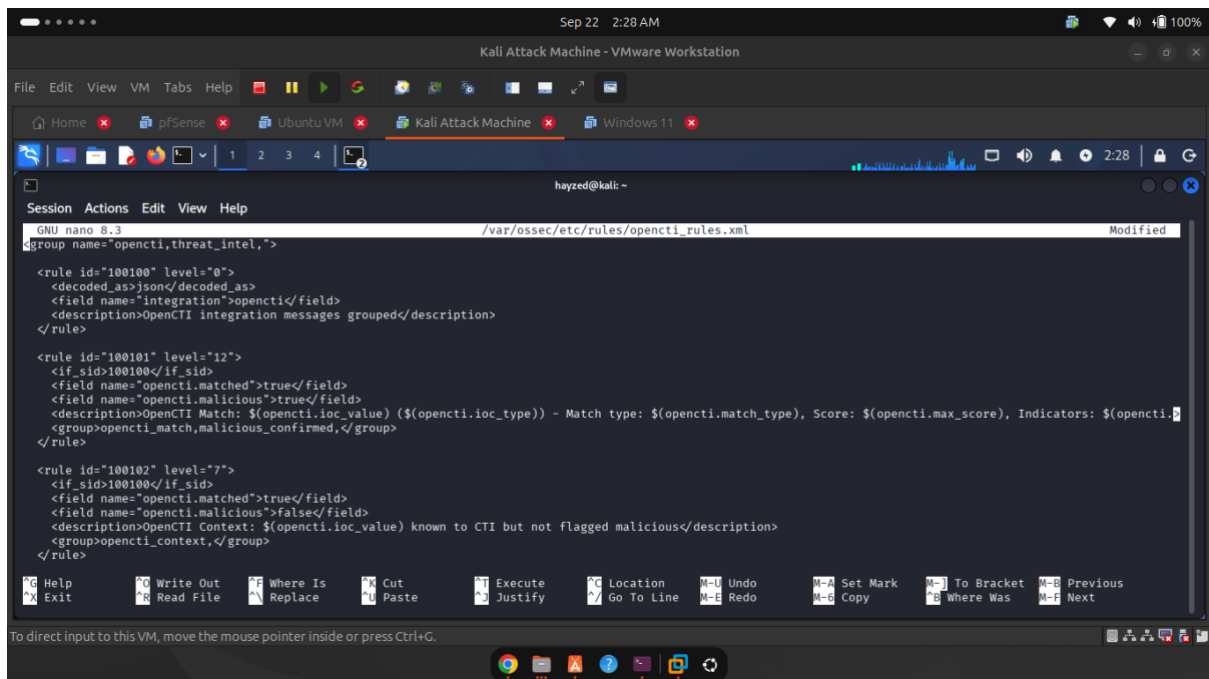


```
GNU nano 8.3 /var/ossec/etc/rules/opencti_rules.xml Modified
<group name="opencti,threat_intel,">

  <rule id="100100" level="0">
    <decoded_as>json</decoded_as>
    <field name="integration">opencti</field>
    <description>OpenCTI integration messages grouped</description>
  </rule>

  <rule id="100101" level="12">
    <if_sid>100100</if_sid>
    <field name="opencti.found">true</field>
    <description>OpenCTI Match: $(opencti.indicator_value) linked to $(opencti.threat_actor) - Confidence: $(opencti.confidence)</description>
    <group>opencti_match,</group>
  </rule>
</group>
```

The final OpenCTI-related detection rules were also validated.



The screenshot shows a Kali Linux virtual machine running in VMware Workstation. The user is editing the file `/var/ossec/etc/rules/opencti_rules.xml` using the nano text editor. The file contains three XML rules for OpenCTI integration. The first rule, `100100`, is a base rule for OpenCTI integration messages. The second rule, `100101`, is a match rule that triggers when an OpenCTI match is confirmed as malicious. The third rule, `100102`, is a context rule that triggers when an OpenCTI match is known to CTI but not flagged as malicious. The terminal window shows the user's prompt `hayzed@kali: ~` and the nano editor's status bar at the bottom.

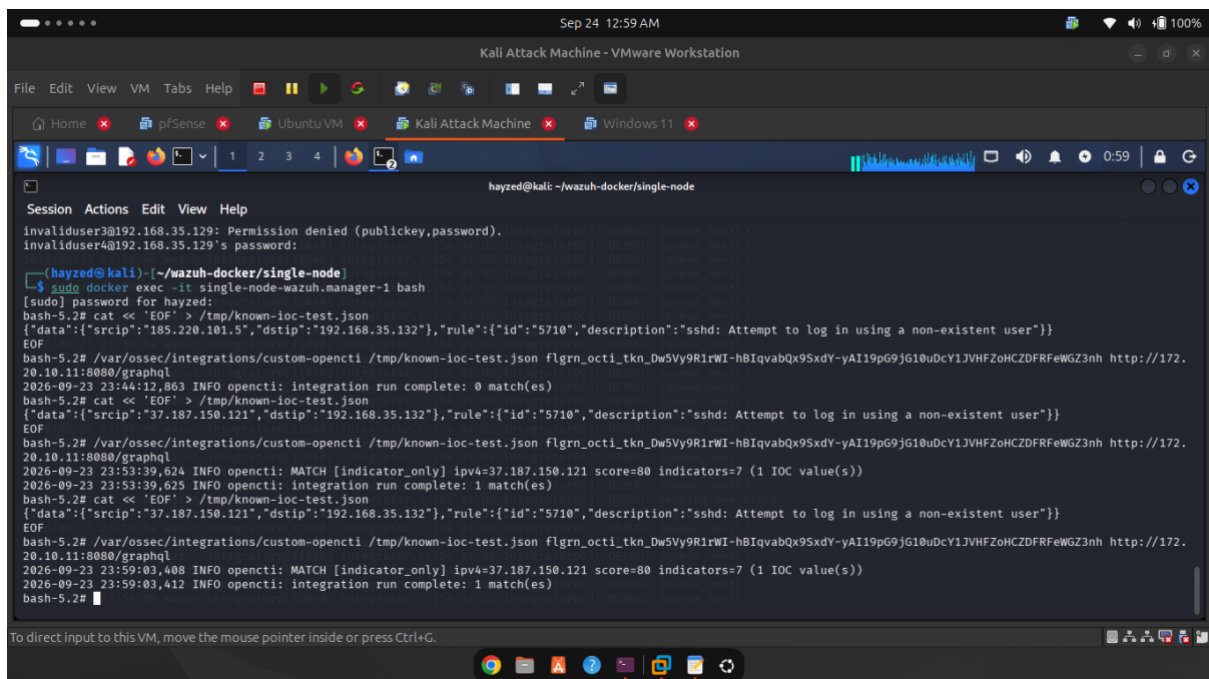
```
GNU nano 8.3 /var/ossec/etc/rules/opencti_rules.xml
<group name="opencti.threat_intel">

<rule id="100100" level="0">
  <decoded_as>json</decoded_as>
  <field name="integration">opencti</field>
  <description>OpenCTI integration messages grouped</description>
</rule>

<rule id="100101" level="12">
  <if_sid>100100</if_sid>
  <field name="opencti.matched">true</field>
  <field name="opencti.malicious">true</field>
  <description>OpenCTI Match: $(opencti.ioc_value) ($(opencti.ioc_type)) - Match type: $(opencti.match_type), Score: $(opencti.max_score), Indicators: $(opencti.ioc_value)</description>
  <group>opencti_match_malicious_confirmed,</group>
</rule>

<rule id="100102" level="7">
  <if_sid>100100</if_sid>
  <field name="opencti.matched">true</field>
  <field name="opencti.malicious">false</field>
  <description>OpenCTI Context: $(opencti.ioc_value) known to CTI but not flagged malicious</description>
  <group>opencti_context,</group>
</rule>
```

A controlled manual test confirmed that Wazuh events could be submitted to the OpenCTI integration for IOC matching.

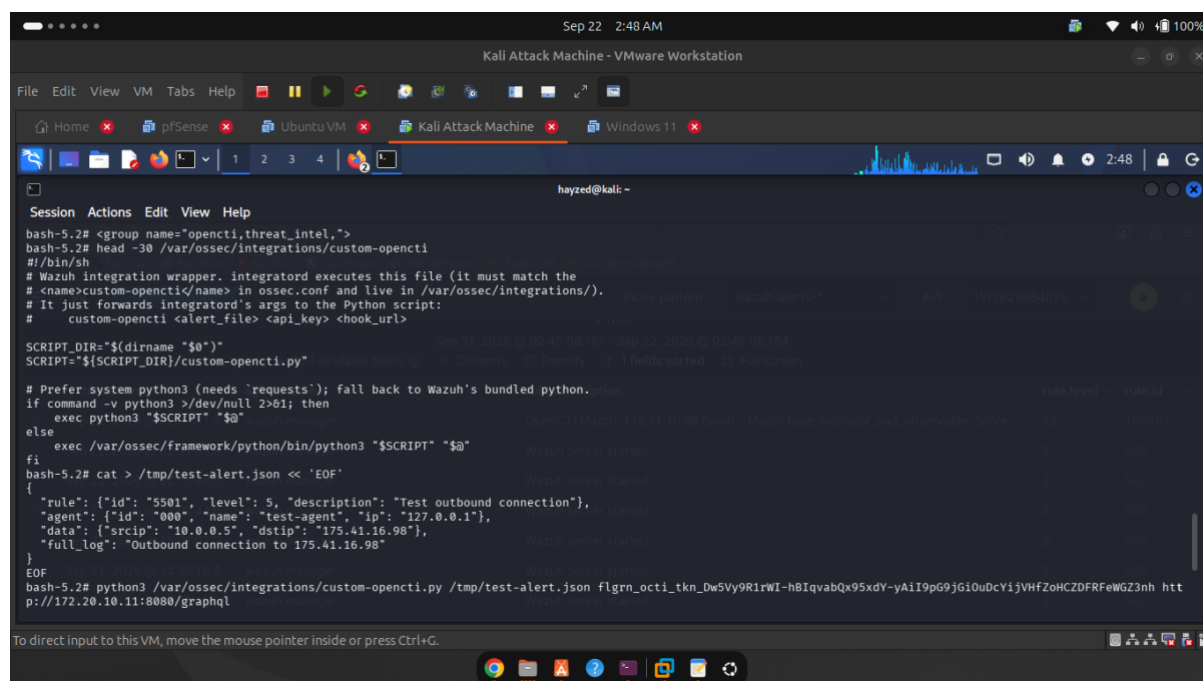


The screenshot shows a Kali Linux virtual machine running in VMware Workstation. The user is in a terminal window, running a series of commands to test the OpenCTI integration. The commands include running `docker exec` to execute a command in a container, and `cat` to view the contents of a file. The output shows the results of the integration run, including the number of matches and the indicators found. The terminal window shows the user's prompt `hayzed@kali: ~/wazuh-docker/single-node` and the nano editor's status bar at the bottom.

```
invaliduser3@192.168.35.129: Permission denied (publickey,password).
invaliduser4@192.168.35.129's password:

(hayzed@kali) (~/.wazuh-docker/single-node)
$ sudo docker exec -it single-node-wazuh.manager-1 bash
[sudo] password for hayzed:
bash-5.2# cat << 'EOF' > /tmp/known-ioc-test.json
{"data":{"srcip":"185.220.101.5","dstip":"192.168.35.132"},"rule":{"id":"5710","description":"sshd: Attempt to log in using a non-existent user"}}
EOF
bash-5.2# /var/ossec/integrations/custom-opencti /tmp/known-ioc-test.json flgrn_octi_tkn_Dw5Vy9R1rWI-hBIqvabQx9SxdY-YAI19pG9jG10uDcY1JVHFZohCZDFRFeWGZ3nh http://172.
20.10.11:8080/graphql
2026-09-23 23:44:12,863 INFO opencti: integration run complete: 0 match(es)
bash-5.2# cat << 'EOF' > /tmp/known-ioc-test.json
{"data":{"srcip":"185.220.101.5","dstip":"192.168.35.132"},"rule":{"id":"5710","description":"sshd: Attempt to log in using a non-existent user"}}
EOF
bash-5.2# /var/ossec/integrations/custom-opencti /tmp/known-ioc-test.json flgrn_octi_tkn_Dw5Vy9R1rWI-hBIqvabQx9SxdY-YAI19pG9jG10uDcY1JVHFZohCZDFRFeWGZ3nh http://172.
20.10.11:8080/graphql
2026-09-23 23:53:39,624 INFO opencti: MATCH [indicator_only] ipv4=37.187.150.121 score=80 indicators=7 (1 IOC value(s))
2026-09-23 23:53:39,625 INFO opencti: integration run complete: 1 match(es)
bash-5.2# cat << 'EOF' > /tmp/known-ioc-test.json
{"data":{"srcip":"37.187.150.121","dstip":"192.168.35.132"},"rule":{"id":"5710","description":"sshd: Attempt to log in using a non-existent user"}}
EOF
bash-5.2# /var/ossec/integrations/custom-opencti /tmp/known-ioc-test.json flgrn_octi_tkn_Dw5Vy9R1rWI-hBIqvabQx9SxdY-YAI19pG9jG10uDcY1JVHFZohCZDFRFeWGZ3nh http://172.
20.10.11:8080/graphql
2026-09-23 23:59:03,408 INFO opencti: MATCH [indicator_only] ipv4=37.187.150.121 score=80 indicators=7 (1 IOC value(s))
2026-09-23 23:59:03,412 INFO opencti: integration run complete: 1 match(es)
bash-5.2#
```

A simulated event was also used to validate the enrichment process.



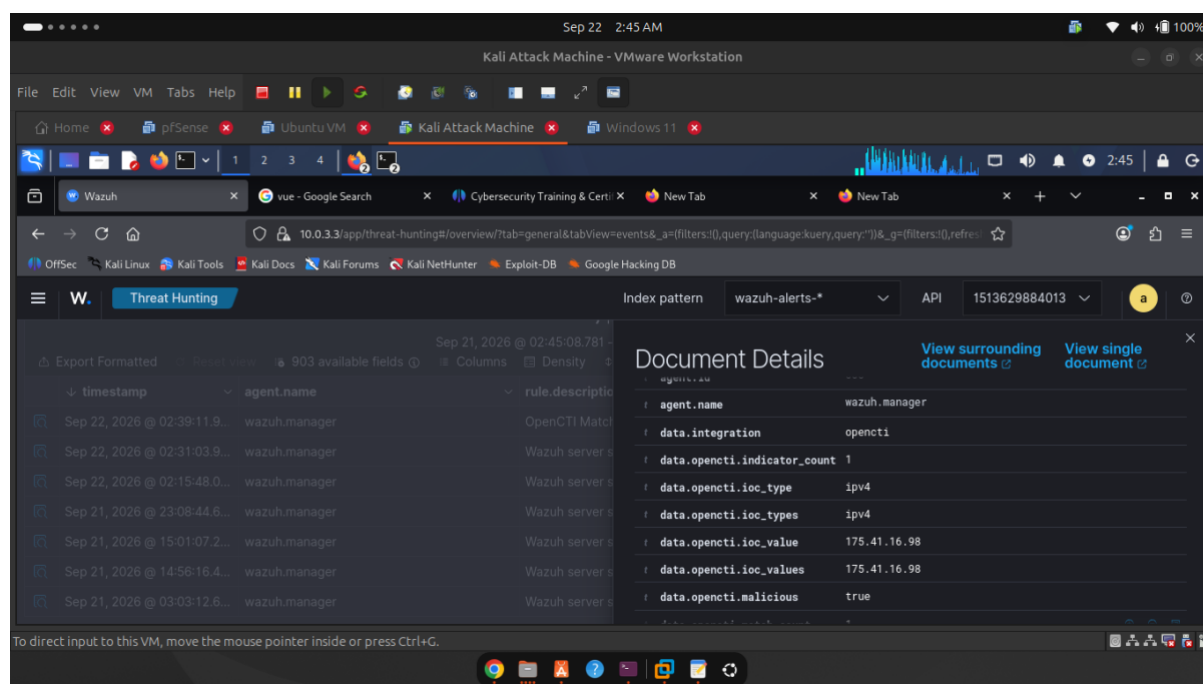
```
bash-5.2# <group name="openciti,threat_intel,">
bash-5.2# head -30 /var/ossec/integrations/custom-openciti
#!/bin/sh
# Wazuh integration wrapper, integrator executes this file (it must match the
# <name>custom-openciti</name> in ossec.conf and live in /var/ossec/integrations/).
# It just forwards integrator's args to the Python script:
#   custom-openciti <alert_file> <api_key> <hook_url>

SCRIPT_DIR="$(dirname "$0")"
SCRIPT="${SCRIPT_DIR}/custom-openciti.py"

# Prefer system python3 (needs 'requests'); fall back to Wazuh's bundled python.
if command -v python3 >/dev/null 2>&1; then
    exec python3 "$SCRIPT" "$@"
else
    exec /var/ossec/framework/python/bin/python3 "$SCRIPT" "$@"
fi

bash-5.2# cat > /tmp/test-alert.json << 'EOF'
{
  "rule": { "id": "5501", "level": 5, "description": "Test outbound connection",
    "agent": { "id": "000", "name": "test-agent", "ip": "127.0.0.1",
      "data": { "srcip": "10.0.0.5", "dstip": "175.41.16.98" },
      "full_log": "Outbound connection to 175.41.16.98"
    }
  }
}
EOF
bash-5.2# python3 /var/ossec/integrations/custom-openciti.py /tmp/test-alert.json flgrn_oci_tkn_Dw5Vy9R1rWI-hBIqvabQx95xdY-yA1I9pG9jGiOuDeYjVHFzOhCZDFRFeWGZ3nh http://172.20.10.11:8080/graphql
```

The integration successfully connected Wazuh events with OpenCTI threat intelligence.



4.4 Event-Driven Threat Intelligence Enrichment

The integration was further validated using both negative and positive IOC tests.

For the negative test, the source IP 185.220.101.5 was submitted to the integration. The integration completed without errors and returned:

integration run complete: 0 match(es)

This confirmed that the integration could process an IOC without generating a match when the indicator was not found.

For the positive test, the source IP 37.187.150.121 was submitted. The integration returned:

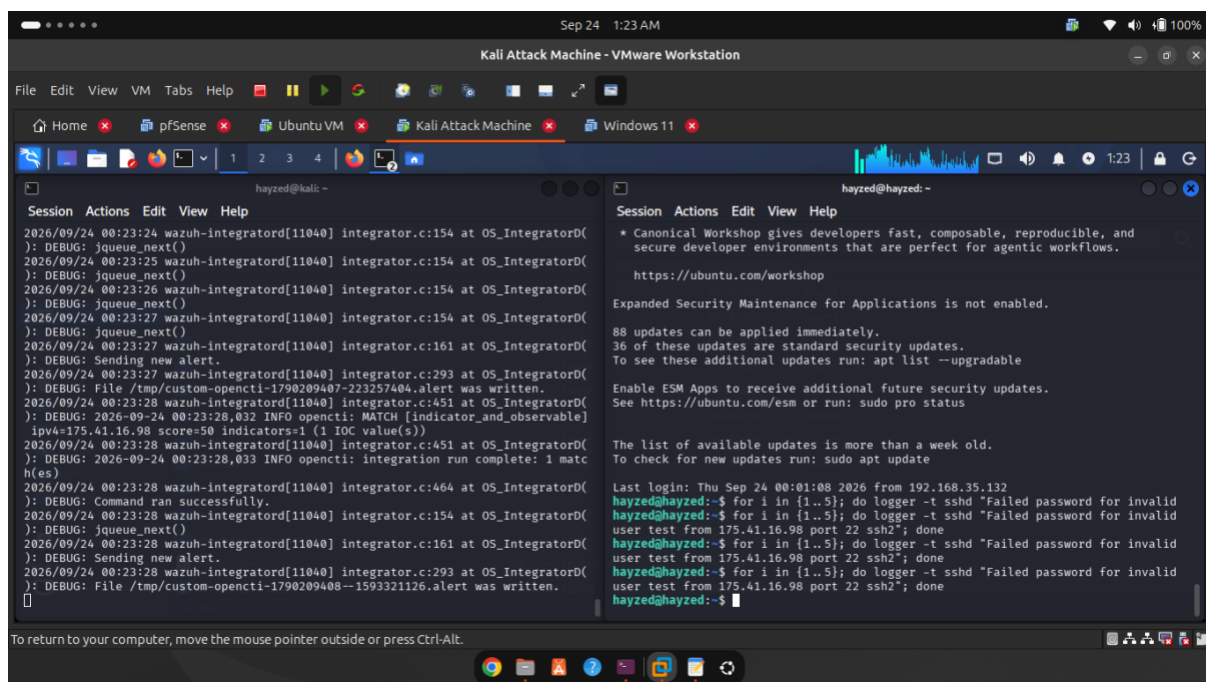
MATCH [indicator_only] ipv4=37.187.150.121 score=80 indicators=7 (1 IOC value(s))

integration run complete: 1 match(es)

This demonstrated that the pipeline could successfully identify a matching indicator and retrieve associated OpenCTI information.

The positive test was repeated and produced the same successful match.

The event-driven process was then tested using a simulated brute-force event involving 175.41.16.98. The event was detected by Wazuh and processed through the OpenCTI integration.



The screenshot shows a Kali Linux virtual machine window titled "Kali Attack Machine - VMware Workstation". The window has a menu bar (File, Edit, View, VM, Tabs, Help) and a toolbar. Below the toolbar is a taskbar with icons for Home, pfSense, Ubuntu VM, Kali Attack Machine, and Windows 11. The main area is split into two terminal windows. The left terminal, titled "hayzed@kali: ~", shows a log of Wazuh-IntegratorD events. The right terminal, titled "hayzed@hayzed: ~", shows system update information and a simulated brute-force attack log. The log in the left terminal includes timestamps, component names, and a final message: "2026-09-24 00:23:28,033 INFO openciti: integration run complete: 1 match(es)". The right terminal shows a message about Canonical Workshop and a list of available updates, followed by a simulated brute-force attack log with timestamps and IP addresses.

```
2026/09/24 00:23:24 wazuh-integratorD[11040] integrator.c:154 at OS_IntegratorD(
): DEBUG: jqueue_next()
2026/09/24 00:23:25 wazuh-integratorD[11040] integrator.c:154 at OS_IntegratorD(
): DEBUG: jqueue_next()
2026/09/24 00:23:26 wazuh-integratorD[11040] integrator.c:154 at OS_IntegratorD(
): DEBUG: jqueue_next()
2026/09/24 00:23:27 wazuh-integratorD[11040] integrator.c:154 at OS_IntegratorD(
): DEBUG: jqueue_next()
2026/09/24 00:23:27 wazuh-integratorD[11040] integrator.c:161 at OS_IntegratorD(
): DEBUG: Sending new alert.
2026/09/24 00:23:27 wazuh-integratorD[11040] integrator.c:293 at OS_IntegratorD(
): DEBUG: File /tmp/custom-openciti-1790209407-223257404.alert was written.
2026/09/24 00:23:28 wazuh-integratorD[11040] integrator.c:451 at OS_IntegratorD(
): DEBUG: 2026-09-24 00:23:28,032 INFO openciti: MATCH [indicator_and_observable]
ipv4=175.41.16.98 score=80 indicators=1 (1 IOC value(s))
2026/09/24 00:23:28 wazuh-integratorD[11040] integrator.c:451 at OS_IntegratorD(
): DEBUG: 2026-09-24 00:23:28,033 INFO openciti: integration run complete: 1 matc
h(es)
2026/09/24 00:23:28 wazuh-integratorD[11040] integrator.c:464 at OS_IntegratorD(
): DEBUG: Command ran successfully.
2026/09/24 00:23:28 wazuh-integratorD[11040] integrator.c:154 at OS_IntegratorD(
): DEBUG: jqueue_next()
2026/09/24 00:23:28 wazuh-integratorD[11040] integrator.c:161 at OS_IntegratorD(
): DEBUG: Sending new alert.
2026/09/24 00:23:28 wazuh-integratorD[11040] integrator.c:293 at OS_IntegratorD(
): DEBUG: File /tmp/custom-openciti-1790209408--1593321126.alert was written.
2026/09/24 00:23:28 wazuh-integratorD[11040] integrator.c:464 at OS_IntegratorD(
): DEBUG: Command ran successfully.
2026/09/24 00:23:28 wazuh-integratorD[11040] integrator.c:154 at OS_IntegratorD(
): DEBUG: jqueue_next()
2026/09/24 00:23:28 wazuh-integratorD[11040] integrator.c:161 at OS_IntegratorD(
): DEBUG: Sending new alert.
2026/09/24 00:23:28 wazuh-integratorD[11040] integrator.c:293 at OS_IntegratorD(
): DEBUG: File /tmp/custom-openciti-1790209408--1593321126.alert was written.
```

```
* Canonical Workshop gives developers fast, composable, reproducible, and
secure developer environments that are perfect for agentic workflows.

https://ubuntu.com/workshop

Expanded Security Maintenance for Applications is not enabled.

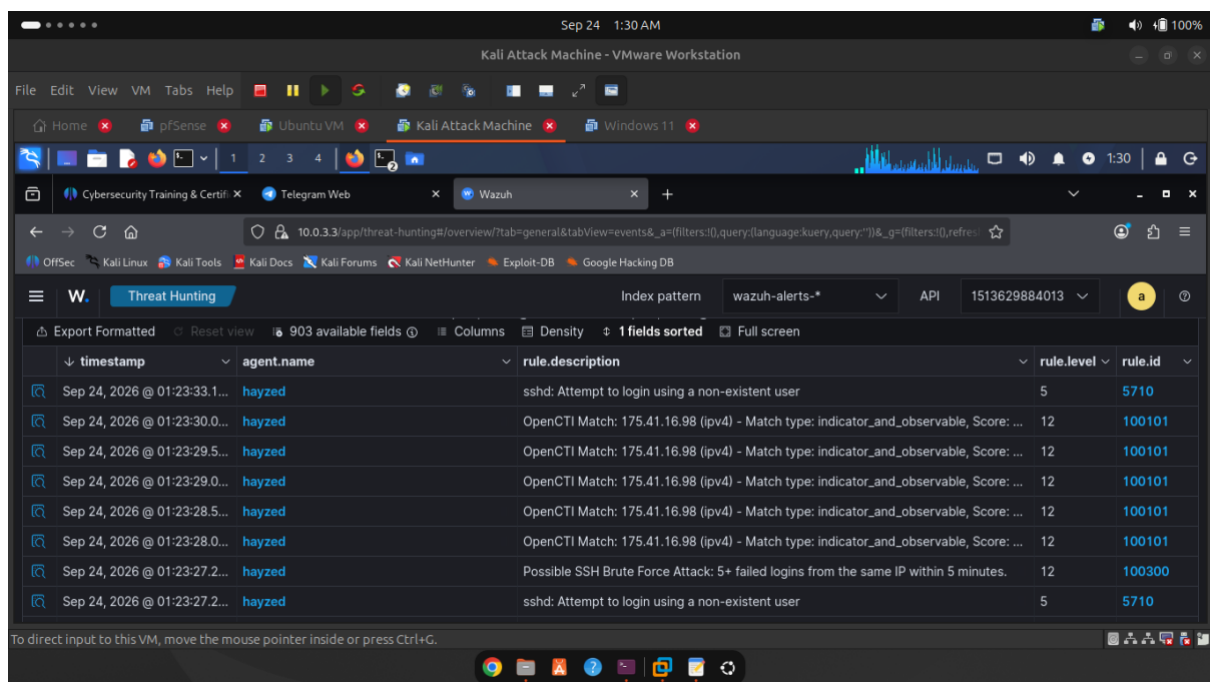
88 updates can be applied immediately.
36 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Thu Sep 24 00:01:08 2026 from 192.168.35.132
hayzed@hayzed:~$ for i in {1..5}; do logger -t sshd "Failed password for invalid
hayzed@hayzed:~$ for i in {1..5}; do logger -t sshd "Failed password for invalid
user test from 175.41.16.98 port 22 ssh2"; done
hayzed@hayzed:~$ for i in {1..5}; do logger -t sshd "Failed password for invalid
user test from 175.41.16.98 port 22 ssh2"; done
hayzed@hayzed:~$ for i in {1..5}; do logger -t sshd "Failed password for invalid
user test from 175.41.16.98 port 22 ssh2"; done
hayzed@hayzed:~$
```

The resulting alert was visible on the Wazuh dashboard



The successful positive and negative tests, together with the live brute-force simulation, demonstrated the complete workflow from Wazuh security event to OpenCTI intelligence lookup and analyst-facing alert.

4.5 Overall Findings

The project successfully added a threat intelligence layer to the existing Wazuh SOC environment.

The final environment included three functioning external threat intelligence sources: URLhaus, MalwareBazaar, and ThreatFox. MITRE ATT&CK information was also made available within OpenCTI.

The Wazuh–OpenCTI integration was successfully tested using both manually generated events and live event-driven processing. The positive IOC test demonstrated that a known indicator could be matched against OpenCTI, while the negative test demonstrated that a non-matching IOC could be processed without generating a match.

The brute-force use case further demonstrated the practical value of combining an existing Wazuh detection with threat intelligence enrichment. The existing detection identified the suspicious authentication activity, while OpenCTI provided additional context about the source IOC.

5. Gaps / Challenges

5.1 MITRE ATT&CK Data Ingestion

One of the initial challenges involved the MITRE ATT&CK connector. The connector was enabled, but the expected intelligence did not initially appear in OpenCTI. Troubleshooting was performed and the connector was reset, after which MITRE ATT&CK information became available.

A second issue occurred when MITRE ATT&CK information was available but the expected technique IDs were not initially displayed. The connector was reset again, after which the IDs became visible.

This demonstrated that successful connector activation did not immediately guarantee that all expected intelligence would be available or displayed correctly.

5.2 MalwareBazaar Connector Configuration

The MalwareBazaar connector initially failed to deliver intelligence into OpenCTI.

The initial configuration used:

```
OPENCTI_URL=http://localhost:8080
```

Because the connector was running in a separate Docker container, localhost referred to the connector container itself rather than the OpenCTI service.

The configuration was corrected to:

```
OPENCTI_URL=http://opencti:8080
```

The working configuration also used the OpenCTI administrator token and MalwareBazaar API key through environment variables.

After the configuration was corrected, MalwareBazaar successfully delivered threat intelligence into OpenCTI.

5.3 AlienVault OTX Connector Failure

AlienVault OTX was initially selected as one of the external intelligence sources. The connector first produced an unsupported time-format error. The time format was adjusted, but the connector subsequently returned a 503 server error.

Because the connector continued to fail, AlienVault OTX was removed from the deployment and ThreatFox was configured as an alternative threat intelligence source.

ThreatFox successfully populated OpenCTI and became one of the three functioning external intelligence sources used in the final environment.

5.4 Event-Driven Integration and Background Trigger Failure

A significant troubleshooting challenge occurred during the transition from manual integration testing to live event-driven processing.

The OpenCTI integration worked when the script was executed manually, but the background Wazuh integration process initially did not appear to trigger the integration when real security events occurred. This required additional troubleshooting because the manual test suggested that the integration itself was functioning.

The issue was eventually traced to the Wazuh integration debugging configuration. The `integrator.debug` setting in `internal_options.conf` was initially set to:

```
integrator.debug=0
```

This prevented useful integration activity from being visible during troubleshooting.

The setting was changed to:

```
integrator.debug=2
```

The Wazuh Manager was then restarted:

```
/var/ossec/bin/wazuh-control restart
```

After this change, the background integration activity became visible in the logs. A real SSH brute-force simulation was subsequently detected, and the Wazuh integration process automatically picked up the event and processed it through the OpenCTI pipeline.

This was an important validation because it confirmed that the final implementation was event-driven rather than dependent solely on manual execution.

5.5 Resource Constraints and System Optimization

A significant challenge during the project was the limited hardware resources available for running the SOC lab. The available RAM had to be shared between the host operating system, virtual machines, Wazuh, OpenCTI, and supporting services.

The Windows environment required relatively high memory resources because of the number of background services running even while the system was idle. To reduce resource consumption, the Linux-based endpoint environment was moved from Windows to Ubuntu. Ubuntu provided a lighter operating environment and consumed less memory while still providing the required security telemetry and monitoring capabilities.

Additional resource optimization was applied to the OpenCTI deployment by limiting the memory available to individual containers. This helped prevent OpenCTI services from consuming excessive system resources and allowed the wider SOC environment to continue operating within the available hardware capacity.

Although the limited resources introduced constraints, using a lighter Ubuntu environment and applying memory limits allowed the project to continue without requiring additional hardware.

5.6 Summary of Challenges

Challenge	Resolution
MITRE ATT&CK initially failed to ingest	Connector was troubleshot and reset
MITRE ATT&CK IDs were initially not displayed	Connector was reset again
MalwareBazaar did not initially deliver data	Docker configuration was corrected, including the OpenCTI service address
AlienVault OTX failed with time-format and later 503 errors	Connector was removed and replaced with ThreatFox
Manual OpenCTI testing worked but background event-driven processing was difficult to troubleshoot	integrator.debug was increased and the Wazuh Manager was restarted
Limited hardware RAM	Ubuntu was used as a lighter endpoint environment and OpenCTI container memory was limited

6. Recommendations

6.1 Protect Integration Credentials

OpenCTI administrator tokens, Wazuh integration tokens, API keys, and other authentication credentials should be protected and excluded from publicly shared configurations and documentation. Environment variables should continue to be used for sensitive values where appropriate, while actual credentials should remain separated from report examples and presentation materials.

6.2 Keep Alternative Threat Intelligence Sources Available

Alternative threat intelligence sources should be maintained as part of the SOC environment. During the project, AlienVault OTX experienced connector issues and was ultimately replaced with ThreatFox. Maintaining alternative sources provides flexibility when an external feed becomes unavailable or cannot be integrated successfully.

6.3 Maintain Regular Connector Health Checks

The OpenCTI connectors should be checked regularly to confirm that threat intelligence is being ingested successfully. This is particularly important because several connector-related issues were encountered during the project, including MITRE ATT&CK ingestion problems and the MalwareBazaar configuration issue. Regular health checks would help identify similar problems earlier and ensure that the threat intelligence layer continues to provide current information.

7. Appendix

Appendix A — OpenCTI Threat Intelligence Connector Configurations

A.1 URLhaus Connector

connector-urlhaus:

image: opencti/connector-urlhaus:latest

environment:

- OPENCTI_URL=http://opencti:8080
- OPENCTI_TOKEN=\${OPENCTI_ADMIN_TOKEN}
- CONNECTOR_ID=4aa7c4d2-0921-4afa-b3e6-c37ce7f6268f
- CONNECTOR_NAME=URLhaus
- CONNECTOR_SCOPE=urlhaus
- CONNECTOR_LOG_LEVEL=info
- URLHAUS_CSV_URL=https://urlhaus.abuse.ch/downloads/csv_recent/
- URLHAUS_IMPORT_OFFLINE=true
- URLHAUS_CREATE_INDICATORS=true
- URLHAUS_INTERVAL=3

restart: always

mem_limit: 200m

A.2 ThreatFox Connector

connector-threatfox:

image: opencti/connector-threatfox:latest

environment:

- OPENCTI_URL=http://opencti:8080
- OPENCTI_TOKEN=\${OPENCTI_ADMIN_TOKEN}
- CONNECTOR_ID=uuid

- CONNECTOR_NAME=ThreatFox
- CONNECTOR_SCOPE=threatfox
- CONNECTOR_LOG_LEVEL=info
- THREATFOX_CSV_URL=https://threatfox.abuse.ch/export/csv/recent/
- THREATFOX_IMPORT_OFFLINE=true
- THREATFOX_CREATE_INDICATORS=true
- THREATFOX_INTERVAL=3

restart: always

mem_limit: 200m

A.3 MalwareBazaar Connector

connector-malwarebazaar:

image: opencti/connector-malwarebazaar-recent-additions:6.5.5

environment:

- OPENCTI_URL=http://opencti:8080
- OPENCTI_TOKEN=\${OPENCTI_ADMIN_TOKEN}
- CONNECTOR_ID=3c9b7c12-8e4a-4d2b-9a1c-5f8e2d4a6b0c
- CONNECTOR_TYPE=EXTERNAL_IMPORT
- CONNECTOR_NAME=MalwareBazaar Recent Additions
- CONNECTOR_SCOPE=malwarebazaar
- CONNECTOR_LOG_LEVEL=info

MALWAREBAZAAR_RECENT_ADDITIONS_API_KEY=\${MALWAREBAZAAR_API_KEY}

- MALWAREBAZAAR_RECENT_ADDITIONS_INTERVAL=600

restart: always

depends_on:

- opencti

Note: Actual authentication tokens and API keys have been redacted.

Appendix B — Wazuh–OpenCTI Integration Installation

The custom OpenCTI integration files were downloaded into the Wazuh integrations directory.

```
wget -O custom-opencti.py https://raw.githubusercontent.com/anyaml7/wazuh-opencti-integration/refs/heads/main/custom-opencti.py
```

```
wget -O custom-opencti https://raw.githubusercontent.com/anyaml7/wazuh-opencti-integration/refs/heads/main/custom-opencti
```

Permissions were configured using:

```
chmod 750 /var/ossec/integrations/custom-opencti.py /var/ossec/integrations/custom-opencti
```

```
chown root:wazuh /var/ossec/integrations/custom-opencti.py  
/var/ossec/integrations/custom-opencti
```

Appendix C — OpenCTI Integration Environment File

An environment file was created to store the Wazuh integration token.

```
cat > /var/ossec/integrations/opencti.env << 'EOF'
```

```
OPENCTI_TOKEN=<REDACTED>
```

```
EOF
```

```
cat > /var/ossec/integrations/opencti.env << 'EOF'
```

```
OPENCTI_TOKEN=<REDACTED>
```

```
EOF
```

Permissions were then restricted:

```
chmod 640 /var/ossec/integrations/opencti.env
```

```
chown root:wazuh /var/ossec/integrations/opencti.env
```

Appendix D — Wazuh Manager Integration Configuration

The OpenCTI integration was added to the Wazuh Manager configuration.

```
<integration>
  <name>custom-opencti</name>
  <hook_url>http://172.20.10.11:8080/graphql</hook_url>
  <api_key>REDACTED</api_key>
  <level>5</level>
  <alert_format>json</alert_format>
</integration>
```

Appendix E — OpenCTI Wazuh Detection Rules

```
<group name="opencti,threat_intel,">
  <rule id="100100" level="0">
    <decoded_as>json</decoded_as>
    <field name="integration">opencti</field>
    <description>OpenCTI integration messages grouped</description>
  </rule>
  <rule id="100101" level="12">
    <if_sid>100100</if_sid>
    <field name="opencti.matched">true</field>
    <field name="opencti.malicious">true</field>
    <description>OpenCTI Match: $(opencti.ioc_value) ($(opencti.ioc_type)) - Match
type: $(opencti.match_type), Score: $(opencti.max_score), Indicators:
$(opencti.indicator_count)</description>
    <group>opencti_match,malicious_confirmed,</group>
  </rule>
```

```

<rule id="100102" level="7">
  <if_sid>100100</if_sid>
  <field name="opencti.matched">true</field>
  <field name="opencti.malicious">false</field>
  <description>OpenCTI Context: $(opencti.ioc_value) known to CTI but not flagged
malicious</description>
  <group>opencti_context,</group>
</rule>
</group>

```

The following rules directory was also added:

```

<rule_dir>/var/ossec/etc/rules</rule_dir>

```

Appendix F — Controlled OpenCTI Integration Test

A controlled Wazuh alert was created for testing:

```

cat > /tmp/test-alert.json << 'EOF'
{
  "rule": {
    "id": "5501",
    "level": 5,
    "description": "Test outbound connection"
  },
  "agent": {
    "id": "000",
    "name": "test-agent",
    "ip": "127.0.0.1"
  },
  "data": {

```

```
"srcip": "10.0.0.5",
"dstip": "175.41.16.98"
},
"full_log": "Outbound connection to 175.41.16.98"
}
EOF
```

The integration was executed using:

```
python3 /var/ossec/integrations/custom-opencti.py \
/tmp/test-alert.json \
<REDACTED_TOKEN> \
http://192.168.351:8080/graphql
```

Appendix G — Negative IOC Test

```
cat << 'EOF' > /tmp/known-ioc-test.json
```

```
{"data":{"srcip":"185.220.101.5","dstip":"192.168.35.132"},"rule":{"id":"5710","description":"sshd: Attempt to log in using a non-existent user"}}
```

```
EOF
```

The integration was executed using:

```
/var/ossec/integrations/custom-opencti \
/tmp/known-ioc-test.json \
<REDACTED_TOKEN> \
http://192.168.35.1:8080/graphql
```

Observed result:

```
2026-09-23 23:44:12,863 INFO opencti: integration run complete: 0 match(es)
```

Appendix H — Positive IOC Test

```
cat << 'EOF' > /tmp/known-ioc-test.json
```

```
{"data":{"srcip":"37.187.150.121","dstip":"192.168.35.132"},"rule":{"id":"5710","description":"sshd: Attempt to log in using a non-existent user"}}
```

EOF

The integration was executed using:

```
/var/ossec/integrations/custom-opencti \
```

```
/tmp/known-ioc-test.json \
```

```
<REDACTED_TOKEN> \
```

```
http://192.168.35.1:8080/graphql
```

Observed result:

```
2026-09-23 23:53:39,624 INFO opencti: MATCH [indicator_only]  
ipv4=37.187.150.121 score=80 indicators=7 (1 IOC value(s))
```

```
2026-09-23 23:53:39,625 INFO opencti: integration run complete: 1 match(es)
```

Appendix I — Wazuh Integration Debugging

The Wazuh integration debugging level was initially:

```
integrator.debug=0
```

It was changed to:

```
integrator.debug=2
```

The Wazuh Manager was then restarted:

```
/var/ossec/bin/wazuh-control restart
```

This provided additional visibility into the background integration process and assisted in confirming the event-driven OpenCTI workflow

Appendix J — Brute-Force Simulation

The brute-force activity was simulated using:

```
for i in {1..5}; do logger -t sshd "Failed password for invalid user test from 175.41.16.98 port 22 ssh2"; done
```

The generated event was monitored through Wazuh and processed through the OpenCTI enrichment workflow.

Appendix K — Credential Handling

The following sensitive information has intentionally been excluded from this report:

- OpenCTI administrator tokens
- Wazuh/OpenCTI integration tokens
- MalwareBazaar API keys
- Other authentication credentials

Environment-variable references such as `${OPENCTI_ADMIN_TOKEN}` and `${MALWAREBAZAAR_API_KEY}` are retained where they reflect the configuration used in the lab.

Appendix L — Other evidence

Ruleset:

```
<rule_dir>/var/ossec/etc/rules</rule_dir>
```

